

50-Day High-Intensity Engineering Handbook: Friend B (Embedded, DSP, AI & RISC-V)

Zero-Budget (Rs. 0) VS Code + Icarus Verilog + GitHub Collaborative Workflow | 150 Engineering Hours

1. Role Architecture: How Friend A (VLSI) & Friend B (Embedded) Work Together

This project builds two industry-grade hardware-software systems on a strict Rs. 0 budget using VS Code, Icarus Verilog (iverilog), Python, and GCC. Friend A designs the digital hardware in rtl/ and tb/, while Friend B builds the Python Bit-Exact Golden Models, C/FreeRTOS DMA & DSP firmware, INT8 Neural Network Quantization, RISC-V Bare-Metal Bootloader, and MQTT/Grafana Cloud Telemetry in scripts/ and firmware/.

Folder / Area	Owner	Language / Tools	Responsibility
rtl/ & synth/	Friend A (VLSI)	Verilog-2005 / Yosys	Synthesizable NPU, Systolic Array, AXI4, RISC-V SoC, AES/SHA
tb/ & sim/	Friend A + B	Verilog + Hex Vectors	Self-checking testbenches reading Friend B's .hex files
scripts/	Friend B (You)	Python 3 (NumPy/SciPy)	Golden hardware models, INT8 MLP quantization, ELF-to-Hex
firmware/	Friend B (You)	C / FreeRTOS / RISC-V GCC	DMA ring buffers, FFT DSP, SPI/AXI HAL, Secure Bootloader
docs/	Both	Markdown / LaTeX / PDF	Register maps, timing reports, 4-page IEEE whitepaper

2. How Friend B Works on the Project Remotely via GitHub

You do NOT need to be on the same laptop! All code lives in a shared GitHub repository (tensor-edge-accelerator). Because Friend A works in rtl/ and tb/ while Friend B works in scripts/ and firmware/, you will NEVER overwrite each other's files (zero merge conflicts).

```
# Step 1 (One-Time Setup on Friend B's Laptop):
git clone https://github.com/<your-username>/tensor-edge-accelerator.git
cd tensor-edge-accelerator

# Step 2 (Every Morning Before Starting Work - Pull Friend A's Latest Hardware):
git pull origin main

# Step 3 (After Completing Your Daily 3-Hour Tasks in scripts/ and firmware/):
git add scripts/ firmware/ sim/*.hex
git commit -m "Day X [Embedded]: Added golden model and DMA firmware"
git push origin main
```

3. Free Tools Friend B Should Install on Their Laptop (<250 MB Total)

Tool	Purpose for Friend B	Install Command (Windows Winget / Pip)
VS Code + Git	Code editor & daily GitHub sync	winget install Microsoft.VisualStudioCode Git.Git
Python 3.10+	Golden models, FFT, INT8 Quantization	pip install numpy scipy scikit-learn matplotlib paho-mqtt
GCC (MinGW-w64)	Compile & test C DMA / DSP / AES code	winget install BrechtSanders.WinLibs.POSIX.UCRT
Icarus Verilog	Run Friend A's hardware with your .hex vectors	winget install IcarusVerilog.IcarusVerilog
xPack RISC-V GCC	Phase 2 (Days 25-50) RISC-V cross-compiler	Download xpack-riscv-none-embed-gcc zip

4. Phase 1 Curriculum for Friend B: Edge AI Tensor Accelerator (Days 1-25)

Each module below lists (1) The Core Theory You Must Learn First, (2) Your Exact Daily Code Deliverables, and (3) How Your Output Connects to Friend A's Verilog Hardware.

Module 1 (Days 1-5): Fixed-Point Arithmetic, Circular DMA, FFT DSP & INT8 Quantization

Theory to Study: (1) Two's complement & Q1.7 fixed-point representation (multiplying float by 128, arithmetic right shift >> 7, saturation at +127/-128). (2) Lock-free Circular Ring Buffers using head/tail pointers & power-of-2 bitmasking ((head + 1) & 255). (3) Fast Fourier Transform (FFT) spectral energy bins for bearing vibration faults. (4) Symmetric INT8 neural network weight quantization: $Scale\ S = \max(|W|) / 127$.

Day	Deliverable File(s)	Engineering Objective
Day 1	scripts/golden_model.py & firmware/dma_ring_buffer.c	Generate 65,536 Q1.7 hex test vectors + C circular DMA buffer.
Day 2	firmware/mpu6050_fifo_sim.c	Stream NASA/CWRU bearing fault data at 1 kHz into the DMA ring buffer.
Day 3	firmware/fixed_fft_dsp.c	Compute 128-point fixed-point FFT and extract 16 Q1.7 spectral features.
Day 4	scripts/train_quantize_mlp.py	Train 3-layer MLP (16->32->16->1), quantize weights to INT8 .hex files.
Day 5	firmware/spi_packet_crc16.c	Frame packets [0xA55A CMD ADDR LEN PAYLOAD CRC16] in C.

Module 2 (Days 6-10): FreeRTOS Multi-Tasking, File-Mailbox Co-Sim & NPU C Driver

Theory to Study: (1) Real-Time Operating Systems (FreeRTOS task priorities, preemption, mutexes, binary semaphores, priority inversion). (2) Hardware Abstraction Layers (HAL) & memory-mapped register drivers in C (volatile uint32_t). (3) Co-simulation mailbox bridge: C writes tx_Command.hex -> iverilog computes -> C reads rx_Status.hex.

Day	Deliverable File(s)	Engineering Objective
Day 6	firmware/freertos_tasks.c	Implement 4 prioritized tasks (SensorRead, DSP_Filter, NPU_Comm, Telemetry).
Day 7	scripts/cosim_bridge.py	Bidirectional pipe connecting C firmware frames to Friend A's 4x4 Systolic Array.
Day 8	firmware/triple_buffer_cli.c	Triple-buffer pool + interactive CLI to inject bearing fault anomalies.
Day 9	firmware/npu_driver.c & npu_driver.h	Implement npu_init(), npu_load_weights(), npu_infer(), and 50us timeout reset.
Day 10	scripts/stress_test_10k.py	Run 10,000 continuous co-simulated inferences and verify 0 bit errors.

Module 3-5 (Days 11-25): AXI4-Lite HAL, MQTT Cloud Telemetry, Benchmarking & CI/CD

Theory to Study: (1) AMBA AXI4-Lite register map (CONTROL=0x00, STATUS=0x04, LAYER_CFG=0x08, CYCLES=0x0C) and AXI4-Stream bursts. (2) MQTT Publish/Subscribe telemetry architecture & JSON formatting. (3) Amdahl's Law & Hardware Speedup calculation (CPU software cycles vs. NPU hardware cycles).

Day	Deliverable File(s)	Engineering Objective
Days 11-12	firmware/axi_lite_hal.c	Map C register reads/writes to AXI4-Lite and package 3-layer weights for AXI-Stream.
Days 13-14	scripts/mqtt_grafana_pub.py	Publish live anomaly scores & cycle counts every 100ms to Mosquitto/Grafana.
Days 15-18	scripts/roc_snr_sweep.py	Sweep noise (-5dB to +20dB) on bearing dataset and plot real-time fault jump (<15 to >85).
Days 19-20	scripts/benchmark_speedup.py	Measure pure C CPU inference time vs Hardware NPU cycles (>18x speedup chart).
Days 21-25	.github/workflows/ci.yml	Set up GitHub Actions automated regression + install xPack RISC-V GCC for Phase 2.

5. Phase 2 Curriculum for Friend B: RISC-V Crypto SoC (Days 26-50)

In Phase 2 (secure-riscv-crypto-soc), Friend A builds a 32-bit RISC-V SoC with Wishbone AES-128 and SHA-256 hardware accelerators. Friend B writes the bare-metal RISC-V startup assembly (crt0.s), custom linker script (linker.ld), C cryptographic drivers, Hardware Root-of-Trust Secure Bootloader, and Cloud Decryption Gateway.

Day	Deliverable File(s)	Theory & Engineering Objective
Day 26	firmware/aes128_sw.c	Study FIPS-197. Implement SubBytes, ShiftRows, MixColumns, AddRoundKey in C (NIST vec
Day 27	sim/run_picorv32.sh	Compile open-source PicoRV32 (RV32IM) in iverilog and test basic RISC-V instructions.
Day 28	firmware/linker.ld & crt0.s	Learn bare-metal boot: Set stack pointer (sp), clear .bss, map FLASH (0x0) & SRAM (0x20000
Day 29	scripts/elf2hex.py	Convert compiled RISC-V ELF binaries into Verilog \$readmemh hex memory images.
Day 30	firmware/bench_aes_sw.c	Use RISC-V rdcycle CSR instruction to count software AES-128 cycles (~4,200 cycles/block).
Day 31	firmware/aes_hw.h & aes_hw.c	Write Wishbone C HAL for AES coprocessor at 0x40000000 (CTRL, STATUS, KEY, DATA_IN
Day 32	firmware/march_c_ram_test.c	Implement March C- memory test algorithm to validate 32KB SRAM across Wishbone bus.
Day 33	firmware/mini_printf.c	Write zero-bloat bare-metal print_str(), print_hex(), and print_dec() over Wishbone UART (0x50
Day 34-35	scripts/uart_mqtt_bridge.py	Boot full RISC-V SoC in iverilog; capture UART ciphertext output and bridge to MQTT broker.
Day 36-37	firmware/sha256_hw.c	Write C reference SHA-256/HMAC (RFC 2202) + Wishbone hardware driver at 0x70000000.
Day 38	firmware/secure_bootloader.c	Compute hardware SHA-256 of app binary on boot; halt CPU if hash != Hardware Root-of-Trus
Day 39-41	scripts/cloud_decrypt_verifier.py	Frame [TIMESTAMP NONCE PAYLOAD HMAC], encrypt with AES-128, verify & decrypt in
Day 42-45	scripts/plot_crypto_speedup.py	Benchmark Hardware (11 cycles) vs Software (~4,200 cycles) = ~93x speedup + constant-time
Day 46-50	docs/IEEE_Whitepaper_4Page.pdf	Co-author 4-page IEEE paper, finalize GitHub portfolio, submit to IIT Madras/IITSc/IITB/IITD.

6. Friend B's Immediate Day 1 Checklist (Start Today!)

1. Clone the GitHub repository onto your laptop and open it in VS Code.
2. Run **python scripts/golden_model.py** to generate the 65,536 Q1.7 hex vectors in **sim/golden_vectors.hex**.
3. Compile and test the Circular DMA Ring Buffer: **gcc firmware/dma_ring_buffer.c -o sim/dma_test && ./sim/dma_test**.
4. Commit and push your Day 1 progress to GitHub!